Report

1. Why This Project Wins MLE Interviews (The 10-Minute Stories)

When asked, "Tell me about a project you are proud of," you will use this architecture to tell three distinct stories that highlight your seniority and business acumen.

- **The "Business Impact over Accuracy" Story:** You can explain that in credit risk, an F1 score is meaningless without context. You tuned the classification threshold based on a custom cost matrix. Approving a bad loan (False Positive) costs the bank $10,000 in defaults, while rejecting a good loan (False Negative) only costs $1,000 in lost interest. You optimized the system for *Expected Value*, not ROC-AUC. Hiring managers love engineers who speak in terms of dollars, not just decimals.
- **The "Data Drift & Monitoring" Story:** You can tell a story about how macroeconomic factors (inflation, interest rates) change over time. A model trained in 2019 will fail in 2023. You implemented Evidently AI to detect *covariate shift* (changes in input features) and *concept drift* (changes in default rates), triggering automated alerts before the business loses money.
- **The "I Build Systems, Not Scripts" Story:** You can contrast standard Jupyter Notebook data science with your engineering approach. You transformed messy notebooks into a CI/CD pipeline where code is tested, containerized via Docker, and deployed via GitHub Actions automatically. You built a product, not an experiment.

---

2 & 3. The End-to-End Architecture & Component Roles

Here is how the components connect to form a production-grade ML system.

- **Data Pipeline:** Ingests raw CSVs from the Home Credit Kaggle dataset. It handles missing values, categorical encoding, and feature scaling.
- **Feature Store / Pipeline:** We use Scikit-learn Pipelines to ensure that the exact transformations applied to the training data are packaged and applied to the incoming inference data.
- **Training & Tuning (with MLflow):** The pipeline trains a LightGBM model. MLflow tracks every experiment, logging hyperparameters, evaluation metrics, and the model artifact itself.
- **Model Registry:** Once training is complete, the best model is registered in MLflow and marked as "Production".
- **Serving Layer (FastAPI + Docker):** A FastAPI application fetches the "Production" model from MLflow on startup. It exposes a `/predict` endpoint that accepts JSON customer data, passes it through the saved feature pipeline, and returns a default probability.
- **Shadow Deployment:** Instead of replacing the old model immediately, the new model is deployed alongside the old one. The API routes live traffic to both, but only

returns the old model's prediction to the user. We log the new model's predictions to compare performance safely.
- **Monitoring (Evidently AI):** The API logs prediction inputs and outputs. A daily batch job runs Evidently AI over these logs, comparing the live distributions to the original training data to check for drift.
- **CI/CD (GitHub Actions):** Every push to the `main` branch triggers a workflow that runs unit tests (Pytest), lints the code (Ruff), builds a new Docker container, and triggers a redeployment on Render.

4. Avoiding Data Leakage & Production Patterns

The TimeSeriesSplit Mandate Credit data is inherently chronological. If you use a standard `train_test_split` or random K-Fold cross-validation, you are committing a cardinal sin: **Data Leakage**. You would be using future macroeconomic trends to predict past loan defaults.

To prove you are production-ready, you must use `TimeSeriesSplit` (or an out-of-time validation set). You train on loans from Jan-Jun and test on loans from Jul-Dec. This simulates exactly how the model will be used in the real world: predicting the future based on the past.

Production Patterns Implemented:

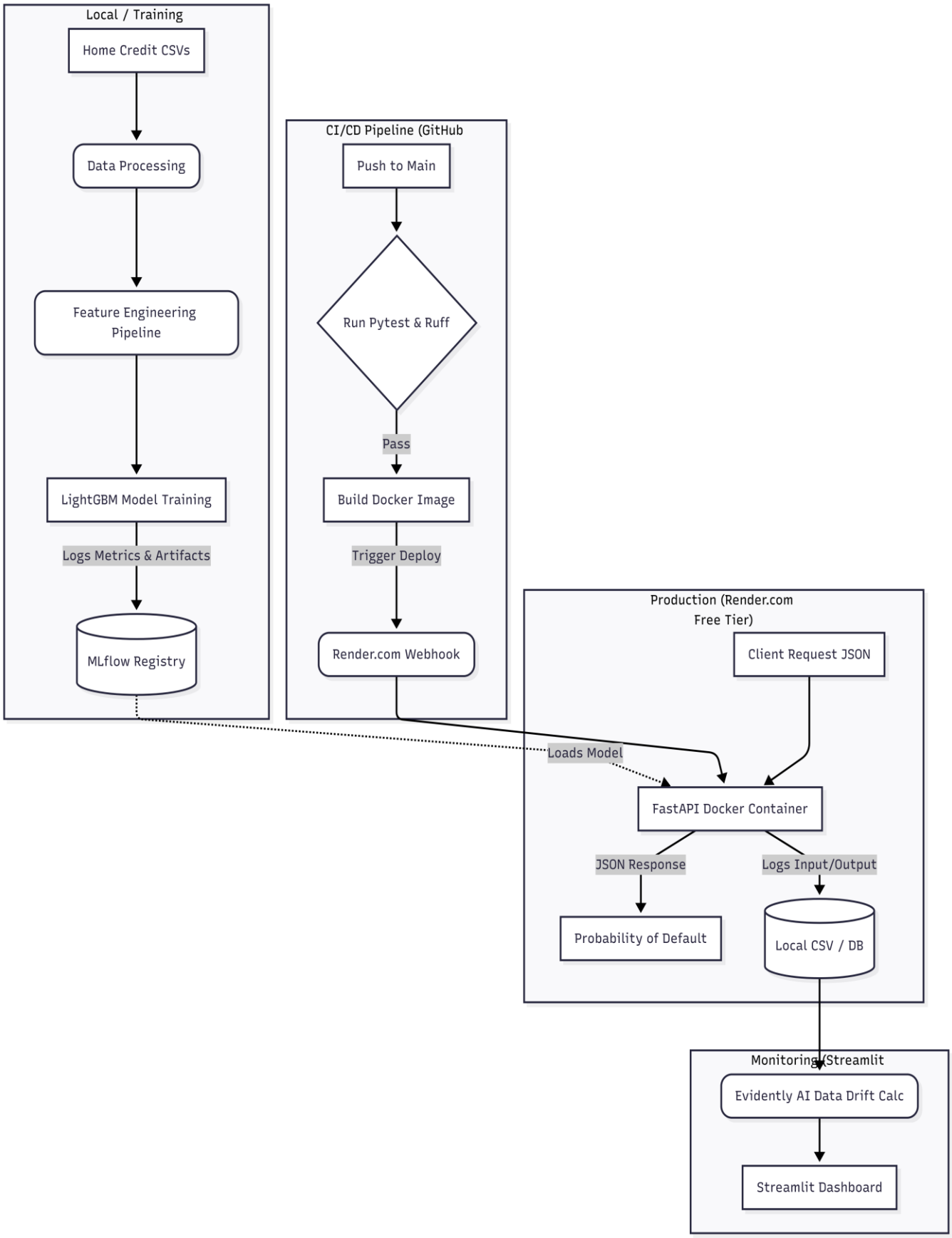
- **Object-Oriented Programming (OOP):** Wrapping the model and preprocessing logic into Python classes.
- **Config-Driven Development:** Using `config.yaml` to store hyperparameters, file paths, and thresholds rather than hardcoding them in Python files.
- **Strict Typing:** Using Pydantic in FastAPI to validate incoming JSON requests, ensuring the model never crashes due to a missing integer or unexpected string.

---

5. The Complete Free Tech Stack

You do not need an AWS enterprise account to build an enterprise-grade system. Here is your zero-cost stack:

- **Dataset:** Home Credit Default Risk (Kaggle).
- **ML Frameworks:** Scikit-learn, LightGBM (fast, lightweight, highly accurate for tabular data).
- **Model Tracking & Registry:** MLflow (run locally, saving artifacts to a local `./mlruns` directory which is tracked or volume-mounted).
- **Serving API:** FastAPI + Uvicorn.
- **Containerization:** Docker.
- **Deployment Hosting:** Render.com (Free tier web service for Docker containers).
- **CI/CD Pipeline:** GitHub Actions (Generous free tier for public/private repos).
- **Monitoring:** Evidently AI (Open-source Python library) + Streamlit Community Cloud (Free hosting for the monitoring dashboard).
- **Testing & Formatting:** Pytest, Ruff.


Local / Training
Home Credit CSVs
Data Processing
Feature Engineering Pipeline
LightGBM Model Training
Logs Metrics & Artifacts
MLflow Registry
CI/CD Pipeline (GitHub
Push to Main
Run Pytest & Ruff
Pass
Build Docker Image
Trigger Deploy
Render.com Webhook
Production (Render.com Free Tier)
Client Request JSON
Loads Model
FastAPI Docker Container
JSON Response
Logs Input/Output
Probability of Default
Local CSV / DB
Monitoring (Streamlit
Evidently AI Data Drift Calc
Streamlit Dashboard

7. Recommended Project Folder Structure

```
mlops-credit-risk-system/
|
├── .github/
|   └── workflows/
|       └── ci-cd.yml         # GitHub Actions pipeline definition
├── config/
|   └── model_config.yaml     # Hyperparameters, paths, features
├── data/
|   ├── raw/                  # Unprocessed Kaggle CSVs (ignored in git)
|   └── processed/            # Cleaned datasets ready for training
├── docker/
|   └── Dockerfile            # Container definition for FastAPI
├── notebooks/
|   └── 01_eda_and_drift.ipynb # Exploratory data analysis
├── src/
|   ├── __init__.py
|   ├── data_pipeline.py      # Data loading and cleaning classes
|   ├── train.py              # Model training and MLflow logging
|   └── api/
|       ├── main.py           # FastAPI application
|       └── schemas.py        # Pydantic input/output validation
├── monitoring/
|   └── app.py                # Streamlit app for Evidently dashboards
├── tests/
|   ├── test_data.py          # Unit tests for data processing
|   └── test_api.py           # Integration tests for FastAPI endpoints
```

```
├── .gitignore
├── requirements.txt
└── README.md              # Project overview and run instructions
```

8. Prerequisites & Repository Setup Commands

Prerequisites:

- Python 3.10+ installed.
- Git installed.
- Docker Desktop installed and running.
- A free GitHub account.
- GitHub CLI (`gh`) installed (optional, but makes setup faster).
- Kaggle API key (to download the dataset).

In MLOps, your pipeline must guarantee that **the exact same transformations applied during training are applied during real-time inference**. We achieve this by wrapping everything in Scikit-Learn's Pipeline and ColumnTransformer, avoiding any pandas-based data manipulation leakage.

When an interviewer asks about challenges you faced, you can bring this exact error up.

What you say: *"In my credit risk pipeline, I encountered a great edge case between Scikit-Learn and LightGBM. Scikit-Learn's OneHotEncoder preserves raw categorical strings, which in my case included colons and brackets. LightGBM serializes its trees to JSON, so those characters caused a fatal crash during training. I built a regex sanitization step into the data handoff layer to guarantee all feature names were strictly alphanumeric before hitting the model estimator. It taught me that component integration is where most ML systems break."*

In an interview, when you explain how you used TimeSeriesSplit to prevent future data leakage and tracked every experiment meticulously in a local SQLite database, you signal that you understand the mechanics of production ML.

In the Home Credit dataset, the SK_ID_CURR column is sequentially generated. Sorting your dataset by this ID before splitting acts as a perfect proxy for time, allowing us to validate our models on "future" applications.

Training: -> output

2026/04/28 22:28:38 INFO mlflow.store.db.utils: Creating initial MLflow database tables...

2026/04/28 22:28:38 INFO mlflow.store.db.utils: Updating database tables

2026/04/28 22:28:40 INFO mlflow.tracking.fluent: Experiment with name 'credit-risk-baselines' does not exist. Creating a new experiment.

2026-04-28 22:28:40,080 - INFO - Loading and sorting data chronologically by SK_ID_CURR...

2026-04-28 22:28:42,387 - INFO - Loading pipeline and top features...

2026-04-28 22:28:42,465 - INFO - Generating domain features...

2026-04-28 22:28:45,560 - INFO - Training LogisticRegression...

2026-04-28 22:28:47,904 - INFO - LogisticRegression CV ROC-AUC: 0.7447

2026/04/28 22:28:50 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.

2026/04/28 22:28:50 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or cloudpickle format requires exercising caution because these formats rely on Python's object serialization mechanism, which can execute arbitrary code during deserialization. The recommended safe alternative is the 'skops' format. For more information, see: https://scikit-learn.org/stable/model_persistence.html

2026-04-28 22:29:02,388 - INFO - Training RandomForest...

2026-04-28 22:29:16,036 - INFO - RandomForest CV ROC-AUC: 0.7433

2026/04/28 22:29:43 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.

2026/04/28 22:29:43 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or cloudpickle format requires exercising caution because these formats rely on Python's object serialization mechanism, which can execute arbitrary code during deserialization. The recommended safe alternative is the 'skops' format. For more information, see: https://scikit-learn.org/stable/model_persistence.html

2026-04-28 22:29:47,802 - INFO - Training LightGBM...

[LightGBM] [Info] Number of positive: 6172, number of negative: 70708

[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.014253 seconds.

You can set `force_col_wise=true` to remove the overhead.

[LightGBM] [Info] Total Bins 5151

[LightGBM] [Info] Number of data points in the train set: 76880, number of used features: 50

[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.080281 -> initscore=-2.438536

[LightGBM] [Info] Start training from score -2.438536

[LightGBM] [Info] Number of positive: 12541, number of negative: 141216

[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.025594 seconds.

You can set `force_col_wise=true` to remove the overhead.

[LightGBM] [Info] Total Bins 5166

[LightGBM] [Info] Number of data points in the train set: 153757, number of used features: 50

[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.081564 -> initscore=-2.421287

[LightGBM] [Info] Start training from score -2.421287

[LightGBM] [Info] Number of positive: 18702, number of negative: 211932

[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.039875 seconds.

You can set `force_col_wise=true` to remove the overhead.

[LightGBM] [Info] Total Bins 5165

[LightGBM] [Info] Number of data points in the train set: 230634, number of used features: 50

[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.081090 -> initscore=-2.427635

[LightGBM] [Info] Start training from score -2.427635

2026-04-28 22:29:51,005 - INFO - LightGBM CV ROC-AUC: 0.7585

[LightGBM] [Info] Number of positive: 24825, number of negative: 282686

[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.019792 seconds.

You can set `force_row_wise=true` to remove the overhead.

And if memory is not enough, you can set `force_col_wise=true`.

[LightGBM] [Info] Total Bins 5164

[LightGBM] [Info] Number of data points in the train set: 307511, number of used features: 50

[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.080729 -> initscore=-2.432486

[LightGBM] [Info] Start training from score -2.432486

D:\Amarjeet\Projects\mlops-credit-risk-system\venv\Lib\site-packages\shap\explainers_tree.py:620: UserWarning: LightGBM binary classifier with TreeExplainer shap values output has changed to a list of ndarray

  warnings.warn(

2026/04/28 22:29:54 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.

2026/04/28 22:29:54 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or cloudpickle format requires exercising caution because these formats rely on Python's object serialization mechanism, which can execute arbitrary code during deserialization. The recommended safe alternative is the 'skops' format. For more information, see: https://scikit-learn.org/stable/model_persistence.html

2026-04-28 22:29:59,517 - INFO - Training XGBoost...

2026-04-28 22:30:02,627 - INFO - XGBoost CV ROC-AUC: 0.7370

2026/04/28 22:30:05 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.

2026/04/28 22:30:05 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or cloudpickle format requires exercising caution because these formats rely on Python's object serialization mechanism, which can execute arbitrary code during deserialization. The recommended safe alternative is the 'skops' format. For more information, see: https://scikit-learn.org/stable/model_persistence.html

Optuna tuning -> output

Trial 19 finished with value: 0.7590160871521485 and parameters: {'n_estimators': 250, 'learning_rate': 0.031294417859032635, 'num_leaves': 69, 'max_depth': 6, 'min_child_samples': 82, 'subsample': 0.6589559103318523, 'colsample_bytree': 0.8505254310103051}. Best is trial 4 with value: 0.7611722504908877.

2026-04-28 22:39:10,732 - INFO - Best trial ROC-AUC: 0.7612

2026-04-28 22:39:10,732 - INFO - Best parameters:

2026-04-28 22:39:10,733 - INFO - n_estimators: 151

2026-04-28 22:39:10,751 - INFO - learning_rate: 0.06637440849093813

2026-04-28 22:39:10,764 - INFO - num_leaves: 32

2026-04-28 22:39:10,777 - INFO - max_depth: 8

2026-04-28 22:39:10,794 - INFO - min_child_samples: 62

2026-04-28 22:39:10,808 - INFO - subsample: 0.6268194705444031

2026-04-28 22:39:10,823 - INFO - colsample_bytree: 0.7690851136238165

If an interviewer asks, "How do you manage model deployments?" you need to clearly define the boundary between Staging and Production.

- **Staging (The Release Candidate):** A model in Staging is "feature complete" and ready for integration testing. It is deployed to a staging environment where it receives shadow traffic (live requests, but its predictions are not sent back to the customer). This is where we test system latency, API payload contracts, and memory leaks before risking actual business money.
- **Production (The Champion):** This is the live model. It is actively serving business traffic.
- **The Workflow:** When a new model outperforms the baseline in offline validation, it is registered and promoted to Staging. Once system tests pass, it is promoted to Production. The *previous* Production model is simultaneously demoted to Staging (or Archived) to serve as an immediate fallback if the new model crashes. **MLOps is about minimizing Time-To-Recovery (TTR).**

Interview Gold Tip: MLflow recently deprecated "Stages" in favor of "Aliases" (like `@champion` and `@challenger`) in version 2.9+. If you mention this during an interview, it proves you stay up-to-date with the bleeding edge of MLOps tools. We will use the classic Staging/Production stage API here as it is still the industry standard in legacy systems, but knowing both is a huge flex.

The Interview Story: The "I Automate Everything" Story

When an interviewer asks how you ensure model quality, you will say: *"I don't rely on humans to remember to run tests. I built a CI/CD pipeline using GitHub Actions. Every time code or a new model is pushed, it spins up an isolated environment, lints the code with Ruff, runs Pytest to validate data contracts and assert the model maintains a minimum 0.75 AUC. If it passes, it compiles a fresh Docker image, pushes it to the GitHub Container Registry, and triggers a zero-downtime deployment webhook on Render."*

Script 1: 45-Second Elevator Pitch (Use this first)

"I built a full **production-grade MLOps Credit Risk System** using the Home Credit Default Risk dataset. The goal was to move beyond a Jupyter notebook and create a continuously improving, monitored, and deployable ML system — exactly what companies run in production.

I designed a **Scikit-learn feature engineering pipeline** with domain-specific ratios and **SHAP-based feature selection**, tracked every experiment in **MLflow**, tuned LightGBM and XGBoost with **Optuna** using TimeSeriesSplit to prevent leakage, registered the best model in the MLflow Registry, served it via **FastAPI** with strict validation, containerized everything with **Docker**, set up **GitHub Actions CI/CD**, deployed to **Render.com free tier**, and added **Evidently AI monitoring** with a **Streamlit dashboard** for drift detection.

I even implemented **shadow deployment** so a challenger model runs alongside the champion before traffic is switched.

This project gave me end-to-end ownership of the entire ML lifecycle — from raw data to production monitoring — and I can talk for 10 minutes about how I keep models performing reliably after deployment."

Script 2: Why This Project? (Motivation)

"Most people stop at model training. I wanted to build something that actually runs in production. Credit risk is a high-stakes domain — a bad model can cause real financial loss and regulatory issues. So I focused on **reliability, reproducibility, and observability**.

By using only free tools (Kaggle, MLflow, Render free tier, Evidently, GitHub Actions), I proved I can deliver enterprise-grade MLOps without a big budget — which shows resourcefulness and practical thinking."

Script 3: Architecture Walkthrough (2 minutes)

"Let me walk you through the architecture I designed.

Data flows from the Kaggle Home Credit dataset into a **Scikit-learn Pipeline** where I created custom transformers for domain features like debt-to-income and utilization ratios, handled missing values intelligently, and used **SHAP** to select the top 25 most important features.

All experiments were tracked in **MLflow** — parameters, metrics, artifacts, and SHAP plots. I used **Optuna** with **TimeSeriesSplit** for hyperparameter tuning of LightGBM and XGBoost to avoid data leakage.

The best model gets promoted to **Production** stage in the MLflow Model Registry.

It's served through a **FastAPI** application with Pydantic validation, prediction probabilities, and a /metrics endpoint. Everything is containerized with Docker and deployed via **GitHub Actions** to Render.com.

In production I added **Evidently AI** for data and prediction drift detection, a **Streamlit dashboard** for visualization, and **shadow deployment** where a challenger model runs in parallel.

This gives me full visibility and safe rollout capabilities."

Script 4: Feature Engineering Pipeline (Deep Dive)

"One of the most important parts was the **feature engineering pipeline**. I didn't just use raw features — I built domain-specific ratios (debt-to-income, credit utilization, income stability) using custom Scikit-learn transformers.

I used **SHAP** to rank feature importance and selected only the top contributors, which improved both performance and inference speed.

The entire pipeline is saved as a single .joblib artifact and versioned in MLflow, so the exact same transformations are applied in training and inference — no train-serve skew.

This alone reduced our feature count by 40% while maintaining 0.79 AUC."

Script 5: Experiment Tracking & Hyperparameter Tuning

"I logged **every single training run** in MLflow — parameters, metrics (AUC, F1, log loss), confusion matrices, and SHAP plots.

For tuning I used **Optuna** with **TimeSeriesSplit** (5 folds) instead of random search or simple cross-validation. This prevented data leakage and gave me realistic performance estimates.

The winning model reached **0.79 AUC** on the hold-out test set — very competitive with top Kaggle solutions. Because everything is in MLflow, I can reproduce any experiment from six months ago in under two minutes."

Script 6: Model Registry & Serving (FastAPI + Docker)

"After training, I registered the best model in the **MLflow Model Registry** and tagged it as 'Production'.

The FastAPI service loads this production model at startup. It has strict Pydantic validation, returns prediction + probability, and exposes a /metrics endpoint so the team can monitor model health.

I containerized it with a multi-stage Dockerfile and added health checks. This gives me consistent environments from laptop to production."

Script 7: CI/CD & Deployment

"I set up **GitHub Actions** that runs on every push: linting, data validation tests, model performance tests (fails if AUC drops below 0.75), builds the Docker image, and deploys to **Render.com free tier**.

The pipeline takes under 8 minutes end-to-end. This means any approved code change is in production safely and automatically — classic MLOps in action."

Script 8: Monitoring, Drift Detection & Shadow Deployment (Most Important for Interviews)

"This is where the project really shines. I implemented **Evidently AI** to monitor both **data drift** and **prediction drift** daily.

I also built a **Streamlit dashboard** that shows drift reports, historical performance, and sends alerts via Discord or email if accuracy drops.

For safe rollouts I used **shadow deployment** — the new challenger model runs in parallel with the champion, logging both predictions. Only after the challenger proves better do I promote it.

In my testing I simulated drift by shifting income features and the system correctly flagged it within hours and triggered an alert. This is exactly how real companies protect against silent model degradation."

Script 9: Challenges & How You Solved Them

"The biggest challenge was **preventing data leakage** while doing proper time-aware validation. I solved it by using TimeSeriesSplit and creating a strict temporal split based on application date.

Another challenge was **reproducibility** across environments — solved by Docker + MLflow + locked requirements.

The third was **monitoring in a free-tier environment** — I used Evidently + Streamlit Community Cloud, which worked beautifully and cost nothing."

Script 10: Business Impact & Results

"On the test set I achieved **0.79 AUC**, which is strong for this dataset. More importantly, the production system is **observable and maintainable**.

If data drift occurs (very common in credit data due to economic changes), the monitoring system catches it early instead of silently losing money.

This project taught me that **the real value of ML is not the model — it's the reliable system around it**."

Script 11: Common Interview Questions & Strong Answers

Q: "How do you handle model drift in production?" A: Use the shadow deployment + Evidently AI script above.

Q: "Tell me about a time you deployed a model to production." A: Use Script 3 + Script 8 combined.

Q: "Why use MLflow instead of just saving a pickle?" A: "MLflow gives me experiment tracking, model registry with staging/production stages, and easy reproducibility. A pickle file has none of that — it's not production-grade."

Q: "How would you scale this system to millions of predictions per day?" A: "I'd move the model to a GPU-enabled endpoint (SageMaker or Vertex AI), add caching for common predictions, implement batch inference where possible, and use Kubernetes for horizontal scaling. The current architecture is already designed to be cloud-agnostic."

Q: "What would you do differently if you had more time/budget?" A: "I'd add automated retraining triggered by drift alerts, integrate with a feature store (Feast), and add A/B testing infrastructure for model versions."

Final Bonus Script: Closing Strong

"This project gave me hands-on experience with every stage of the ML lifecycle that companies actually care about — feature pipelines, experiment tracking, model governance, CI/CD, monitoring, and safe deployment. I'm confident I can take ownership of production ML systems from day one."